

Документация по архитектуре набора инструкций Асс32 (ISA)

ISA Асс32 — это простая архитектура набора инструкций на базе аккумулятора, созданная для учебных целей. В этой документации приводится обзор инструкций, доступных в Асс32 ISA, их синтаксис и семантика.

Обзор архитектуры

Асс32 — это 32-битная аккумуляторная архитектура. Она включает:

- один регистр общего назначения Асс (аккумулятор);
- два флага состояния: переполнение (V) и перенос (C);
- ввод-вывод с отображением в память (Memory-Mapped I/O);
- простой набор инструкций, ориентированный на операции с аккумулятором.

Эта архитектура хорошо подходит для начинающих, чтобы понять основы компьютерной архитектуры без сложности, связанной с множеством регистров.

Комментарии в ассемблерном коде Асс32 обозначаются символом `;`.

Специфичные представления состояния ISA

- `Acc:dec`, `Acc:hex` — регистр Асс.
- `V` — флаг переполнения.
- `C` — флаг переноса.

Инструкции

Размер инструкции: 1 байт для опкода, 4 байта для абсолютного операнда, 2 байта для относительного операнда. Инструкции управления потоком, Load/Store Immediate/Indirect/Addr используют абсолютную адресацию, остальные — относительную.

Инструкции перемещения данных

- **Load Immediate**
 - **Синтаксис:** `load_imm <address>`
 - **Описание:** Загрузить непосредственное значение в аккумулятор.
 - **Операция:** `acc <- <address>`
- **Load**
 - **Синтаксис:** `load <offset>`
 - **Описание:** Загрузить значение из относительного адреса в аккумулятор.
 - **Операция:** `acc <- mem[pc + <offset>]`
- **Store**

- **Синтаксис:** `store <offset>`
- **Описание:** Сохранить значение аккумулятора по относительному адресу.
- **Операция:** `mem[pc + <offset>] <- acc`
- **Load Address**
 - **Синтаксис:** `load_addr <address>`
 - **Описание:** Загрузить значение из заданного адреса в аккумулятор.
 - **Операция:** `acc <- mem[<address>]`
- **Store Address**
 - **Синтаксис:** `store_addr <address>`
 - **Описание:** Сохранить значение аккумулятора по заданному адресу.
 - **Операция:** `mem[<address>] <- acc`
- **Load Acc**
 - **Синтаксис:** `load_acc`
 - **Описание:** Загрузить в аккумулятор значение из адреса, который находится в асс.
 - **Операция:** `acc <- mem[acc]`
- **Store Indirect**
 - **Синтаксис:** `store_ind <address>`
 - **Описание:** Сохранить значение аккумулятора по косвенному адресу.
 - **Операция:** `mem[mem[<address>]] <- acc`

Арифметические инструкции

- **Add**
 - **Синтаксис:** `add <address>`
 - **Описание:** Прибавить к аккумулятору значение из заданного адреса.
 - **Операция:** `acc <- acc + mem[<address>]` и установка флагов **C** и **V**.
- **Subtract**
 - **Синтаксис:** `sub <address>`
 - **Описание:** Вычесть из аккумулятора значение из заданного адреса.
 - **Операция:** `acc <- acc - mem[<address>]` и установка флага **V**.
- **Multiply**
 - **Синтаксис:** `mul <address>`
 - **Описание:** Умножить аккумулятор на значение из заданного адреса.
 - **Операция:** `acc <- acc * mem[<address>]` и установка флага **V**.
- **Divide**
 - **Синтаксис:** `div <address>`
 - **Описание:** Разделить аккумулятор на значение из заданного адреса.

- **Операция:** `acc <- acc / mem[<address>]`
- **Remainder**
 - **Синтаксис:** `rem <address>`
 - **Описание:** Вычислить остаток от деления аккумулятора на значение из заданного адреса.
 - **Операция:** `acc <- acc % mem[<address>]`
- **Clear Overflow**
 - **Синтаксис:** `clv`
 - **Описание:** Сбросить флаг переполнения.
 - **Операция:** `overflow <- 0`
- **Clear Carry**
 - **Синтаксис:** `clc`
 - **Описание:** Сбросить флаг переноса.
 - **Операция:** `carry <- 0`

Побитовые инструкции

- **Shift Left**
 - **Синтаксис:** `shiftl <address>`
 - **Описание:** Сдвинуть аккумулятор влево на число бит из заданного адреса (флаг Carry не используется).
 - **Операция:** `acc <- acc << mem[<address>]`
- **Shift Right**
 - **Синтаксис:** `shiftr <address>`
 - **Описание:** Сдвинуть аккумулятор вправо на число бит из заданного адреса с сохранением знака (флаг Carry не используется).
 - **Операция:** `acc <- acc >> mem[<address>]`
- **Bitwise AND**
 - **Синтаксис:** `and <address>`
 - **Описание:** Выполнить побитовое И между аккумулятором и значением из заданного адреса.
 - **Операция:** `acc <- acc & mem[<address>]`
- **Bitwise OR**
 - **Синтаксис:** `or <address>`
 - **Описание:** Выполнить побитовое ИЛИ между аккумулятором и значением из заданного адреса.
 - **Операция:** `acc <- acc | mem[<address>]`
- **Bitwise XOR**

- **Синтаксис:** `xor <address>`
- **Описание:** Выполнить побитовое исключающее ИЛИ между аккумулятором и значением из заданного адреса.
- **Операция:** `acc <- acc ^ mem[<address>]`
- **Bitwise NOT**
 - **Синтаксис:** `not`
 - **Описание:** Выполнить побитовое НЕ для аккумулятора.
 - **Операция:** `acc <- ~acc`

Инструкции управления потоком

- **Jump**
 - **Синтаксис:** `jmp <address>`
 - **Описание:** Перейти на заданный адрес.
 - **Операция:** `pc <- <address>`
- **Branch if Equal to Zero**
 - **Синтаксис:** `beqz <address>`
 - **Описание:** Перейти на заданный адрес, если аккумулятор равен нулю.
 - **Операция:** `if acc == 0 then pc <- <address>`
- **Branch if Not Equal to Zero**
 - **Синтаксис:** `bnez <address>`
 - **Описание:** Перейти на заданный адрес, если аккумулятор не равен нулю.
 - **Операция:** `if acc != 0 then pc <- <address>`
- **Branch if Greater Than Zero**
 - **Синтаксис:** `bgt <address>`
 - **Описание:** Перейти на заданный адрес, если аккумулятор больше нуля.
 - **Операция:** `if acc > 0 then pc <- <address>`
- **Branch if Less Than Zero**
 - **Синтаксис:** `ble <address>`
 - **Описание:** Перейти на заданный адрес, если аккумулятор меньше нуля.
 - **Операция:** `if acc < 0 then pc <- <address>`
- **Branch if Overflow Set**
 - **Синтаксис:** `bvs <address>`
 - **Описание:** Перейти на заданный адрес, если установлен флаг переполнения.
 - **Операция:** `if overflow == 1 then pc <- <address>`
- **Branch if Overflow Clear**
 - **Синтаксис:** `bvc <address>`

- **Описание:** Перейти на заданный адрес, если флаг переполнения сброшен.
- **Операция:** `if overflow == 0 then pc <- <address>`
- **Branch if Carry Set**
 - **Синтаксис:** `bcs <address>`
 - **Описание:** Перейти на заданный адрес, если установлен флаг переноса.
 - **Операция:** `if carry == 1 then pc <- <address>`
- **Branch if Carry Clear**
 - **Синтаксис:** `bcc <address>`
 - **Описание:** Перейти на заданный адрес, если флаг переноса сброшен.
 - **Операция:** `if carry == 0 then pc <- <address>`
- **Halt**
 - **Синтаксис:** `halt`
 - **Описание:** Остановить машину.